Introduction to Modeling and Verification of Digital Systems

Final Report

ZLATEVSKI Andrej - REVERTER Jordi
Group 11

May 2026





TABLE OF CONTENTS

TABLE OF CONTENTS
1. Introduction
1.1 The RT-Level Design Flow
1.2 Purpose and Scope of the Project
1.3 Hardware Modeling Aspects Illustrated by the Case Study
2. Modeling and Simulation
2.1 The Case Study: Robot Control System
2.2 VHDL Modeling
2.2.1 Controller (Robot entity)
2.2.2 Counter
2.2.3 System
2.3 Test Scenarios and Coverage Analysis
2.4 Simulation Results
3. ASIC Synthesis and Post-Synthesis Verification
3.1 Preparation of the Behavioural Description
3.2 Comparative Synthesis Results
3.3 Post-Synthesis Simulation
4. Validation with PSL Assertions
4.1 Role of Temporal Assertions in the Design Flow
4.2 The Three PSL Properties
4.3 Simulation Results with PSL Assertions Pre-ASIC Synthesis
4.4 Adaptation of PSL Assertions for Post-synthesis Netlists
4.5 Simulation Results after adapting the PSL Assertions for Post-synthesis Netlists
5. FPGA Synthesis
5.1 Adaptations to the VHDL Description
5.2 Synthesis Results
5.3 FPGA simulations
6. Conclusions
Appendix
A.1 Robot Entity and Architecture (pre-synthesis)
A.2 LeonardoSpectrum Synthesis Summary
A.3 FPGA Compilation Summary (Quartus II, EP2C20F484C7)
A.4 Testbench code from lab 3
A.5 Post-ASIC synthesis chronograms, using different types of state encodings

1. Introduction

1.1 The RT-Level Design Flow

Modern digital systems are designed at multiple levels of abstraction, from high-level behavioural specifications down to physical implementation on silicon or programmable logic. The RT-level design flow structures this process into well-defined, verifiable steps, each building on the artefacts produced by the previous one. The following image represents it:

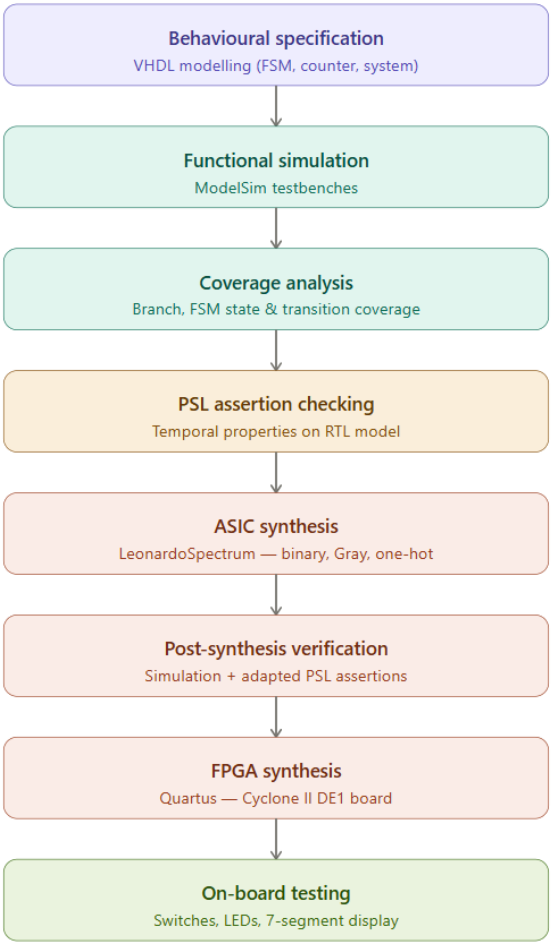


Figure 1: RT level design flow

The flow begins with a behavioural specification: the designer captures the intended functionality using a Hardware Description Language (HDL), in our case VHDL, at the highest level of abstraction. At this stage the description is not yet tied to any technology; it defines what the circuit must do, not how it should be built. The modelling unit is the process: a concurrent statement that reacts to changes in its sensitivity list. Processes fall into two categories depending on their role in the system.

Combinational processes compute new values as pure functions of the current inputs and current state (any change in the sensitivity list immediately triggers a re-evaluation). They must assign every output in every branch to avoid implicitly inferred latches.

Sequential processes capture the notion of memory. They are sensitive only to the clock edge and, optionally, to an asynchronous reset signal. The clock edge acts as a synchronisation barrier: signals assigned inside the process are updated only at that moment, not immediately.

Once the behavioural description by the VHDL code is validated by simulation, it is submitted to an RTL synthesis tool (LeonardoSpectrum for ASIC targets, Quartus for FPGA targets). The tool elaborates the VHDL, infers the hardware primitives that correspond to each process (combinational logic gates for combinational processes, D flip-flops for sequential processes) and maps them onto the cells of a target library. The result is a gate-level netlist. Synthesis can apply various state-encoding strategies for finite state machines, trading off area against timing performance.

Post-synthesis simulation re-runs the original testbenches against the gate-level netlist to confirm that the mapping has preserved the intended behaviour. This step is crucial: although synthesis tools are reliable, subtle issues such as incorrect sensitivity lists, incomplete output assignments (latch inference), or reset-initialisation problems can surface only at this stage.

Formal verification complements simulation throughout the flow. Property Specification Language (PSL) assertions express temporal properties that must hold over all simulation runs. Embedded directly in the VHDL source, they are evaluated by the simulator in real time, flagging violations the moment a counterexample is found. Assertions written at the RTL level must be adapted after synthesis to use the bit-level state encoding chosen by the tool, but the underlying property being verified remains the same.

The final step in our project is FPGA synthesis with Quartus, targeting an Altera Cyclone II DE1 board. This step requires adapting the behavioural description to interface with physical I/O (switches, LEDs, seven-segment displays) and to handle board-specific constraints such as an active-low reset and a 50 MHz clock that must be divided to achieve observable behaviour.

1.2 Purpose and Scope of the Project

The purpose of this project is to model and verify a digital system, a robot with characteristic, predefined behaviours.

The robot control system project serves as a concrete, end-to-end illustration of the RT-level design flow. Each phase of the flow is exercised in a non-trivial way: the FSM and counter components are modelled in VHDL and validated through functional simulation with handwritten testbenches; coverage analysis then measures how thoroughly those testbenches exercise the description, driving the addition of new scenarios. PSL assertions are embedded at the RTL level and re-checked after synthesis, demonstrating how formal properties accompany the design across abstraction boundaries. ASIC synthesis is performed three times (with binary, Gray, and one-hot encoding) making the state-encoding trade-off between area and frequency directly observable rather than purely theoretical. Finally, FPGA synthesis on the Cyclone II DE1 board closes the loop by producing a physically running artefact, confirming that the same VHDL source, with adaptations, can reach two entirely different implementation technologies.

1.3 Hardware Modeling Aspects Illustrated by the Case Study

The case study concretely illustrates the following aspects of hardware modelling:

- Mealy FSM modeling: outputs depend on both the current state and current inputs, requiring careful separation of combinational output logic from the state register.
- Generic parameterisation: the counter component is instantiated twice with different threshold values, demonstrating VHDL generics.
- Structural composition: the top-level System entity wires together Robot and two Count instances, illustrating hierarchical design.

- State encoding trade-offs: binary, Gray, and one-hot encodings are compared experimentally.
- Temporal assertion writing: PSL properties involving sequential operators (until, before, sequences) are written, checked, and adapted after synthesis.
- FPGA adaptation: pin mapping, clock division, seven-segment display encoding, and active-low reset are handled in the top-level entity.

2. Modeling and Simulation

2.1 The Case Study: Robot Control System

The system controls robots that search for food in an arena. The FSM has nine symbolic states:

- idle: initial state entered on reset, immediately transitions to resting.
- resting: the robot rests; the rest counter runs until aboverestth is set to 1.
- randomwalk: the robot searches; the search counter runs. It transitions to movetofood when findfood is set to 1, or to homing when abovesearchth is set to 1.
- movetofood: the robot moves toward detected food. It transitions to grabfood (food output = 1) when closetofood is set to 1, to scanarena when lostfood is set to 1, or to homing when abovesearchth is set to 1.
- scanarena: the robot scans the immediate area. It returns to movetofood if findfood is set to 1, to randomwalk if scantimeup is set to 1, or to homing if abovesearchth is set to 1.
- grabfood: the robot grabs food; waits for success, then transitions to movetohome.
- movetohome: the robot carries food to home; transitions to deposit when athome is set to 1
- deposit: the robot deposits food; transitions to resting (rest output = 1) when success is set to 1.
- homing: emergency return triggered by exceeding the search threshold; transitions to resting (rest output = 1) when athome is set to 1.

The three outputs are: rest (set on transitions leading back to resting), search (set on the transition from resting to randomwalk), and food (set on the transition from movetofood to grabfood).

Two parameterised Count components are instantiated: Count1 (threshold = 4 clock cycles) tracks resting duration; Count2 (threshold = 10 clock cycles) tracks searching duration. Their outputs feed aboverestth and abovesearchth respectively into the Robot.

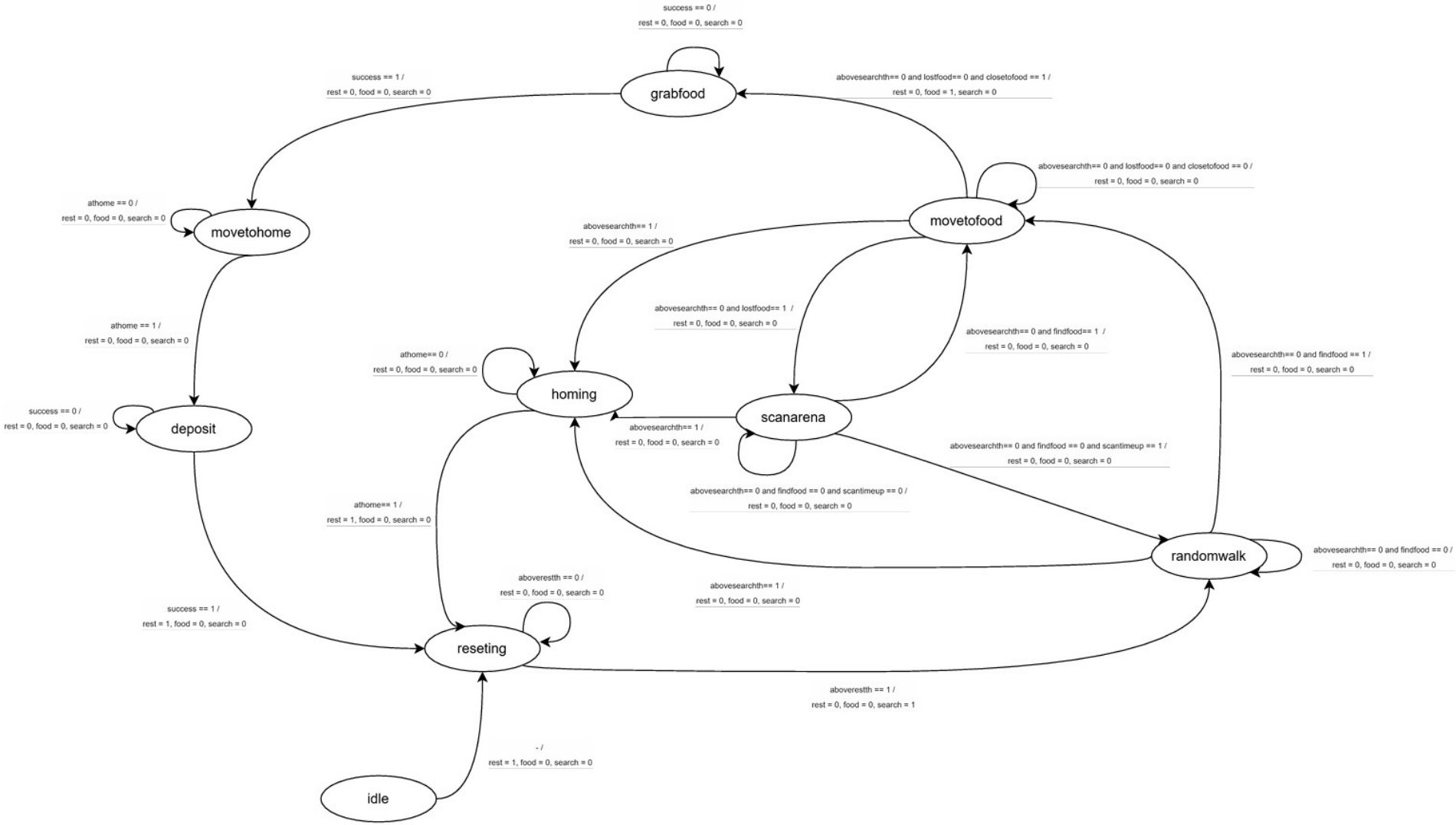


Figure 2: FSM diagram

2.2 VHDL Modeling

2.2.1 Controller (Robot entity)

The controller is described with five concurrent processes, clearly separating the three canonical components of an FSM:

Next-state process: combinational, sensitive to all inputs and the current state. A case statement over state drives the nextstate signal. The threshold conditions (abovesearchth, aboverestth) are evaluated first in each branch to ensure they take priority over other conditions, reflecting the deterministic completion of the automaton.

State-register process: sequential, sensitive to reset and clk. On reset = '1' (active-high in simulation, active-low on FPGA), state reverts to idle. On each rising clock edge state takes the value of nextstate.

Output processes: three separate combinational processes compute rest, search, and food. Each has a sensitivity list containing only the signals it reads. Because every branch assigns a value to the output, no latches are inferred. The rest process is sensitive to state, athome, and success; food depends on state, closetofood, abovesearchth, and lostfood; search depends on state and aboverestth.

This structure is the Mealy machine decomposition. Separating the three output processes is a style choice that makes each sensitivity list smaller and the synthesis result more predictable.

2.2.2 Counter

The counter is itself a small FSM with two states: idle and counting. A generic threshold parameter controls when aboveth is asserted. The internal counter value c is a signal of type natural (later constrained to natural range 0 to 15 for FPGA). The architecture mirrors the Robot structure: a combinational process computes nextstate and nextc; a clocked process stores them. The output assignment aboveth <= '1' when c >= threshold else '0' is a concurrent conditional signal assignment.

2.2.3 System

The System entity instantiates Robot and the two Count components, wiring them according to the specification: rest controls when Count1 starts counting, while search controls when Count2 starts. The outputs from both counters (their “above threshold” signals) are then sent back into the Robot as aboverestth and abovesearchth. The 50 MHz clock is shared by all three components in simulation (on FPGA, the divided 1 Hz clock ck_1Hz is used instead, such that it is interpretable by a human eye).

2.3 Test Scenarios and Coverage Analysis

Six test scenarios were designed to exercise all reachable state transitions:

Test	Scenario	Key objective
1	idle → resting → randomwalk → movetofood → grabfood →	Verify food = 1 under correct conditions; full food-gathering cycle. ;

	movetohome → deposit → resting	needed for PSL property 1 and 2
2	resting → randomwalk → homing → resting	Search threshold exceeded without finding food
3	resting → randomwalk → movetofood → scanarena → homing	Scan arena then homing; athome never set (verify robot stays in homing)
4	resting → randomwalk → movetofood → homing → resting	Search threshold exceeded while tracking food; return to resting
5	resting → randomwalk → movetofood → scanarena → movetofood → homing	Food found, lost, found again
6	resting → randomwalk → movetofood → scanarena → randomwalk → homing	scantimeup triggers return to randomwalk; needed for PSL property 3

The combined coverage report (obtained by merging the six .cov files with vcover) for robot.vhd showed:

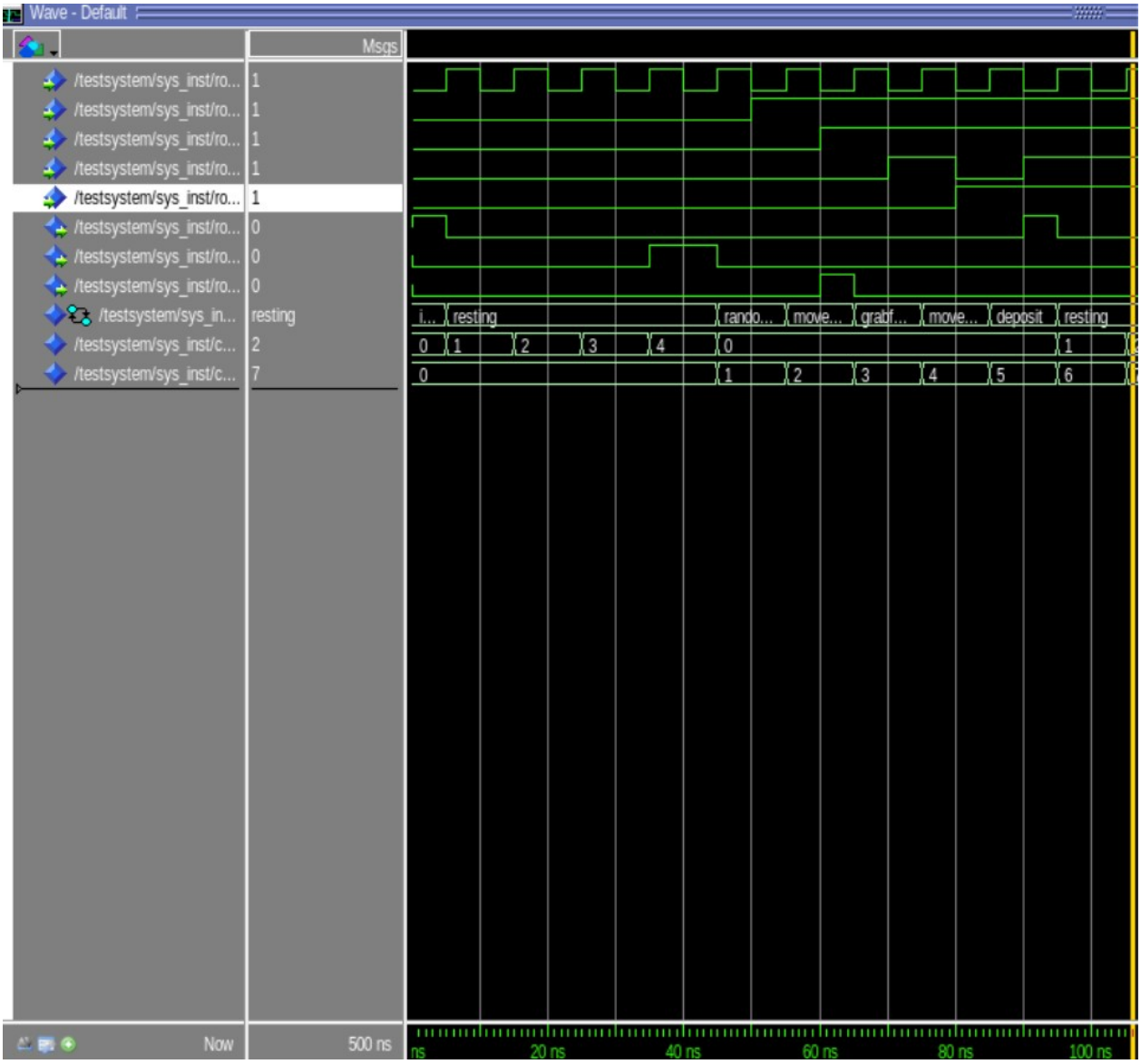
- Statement coverage: 100%
- Branch coverage: 100%
- FSM state coverage: 100% (all nine states visited)
- FSM transition coverage: 66.66% (16 out of 24 transitions covered). The eight uncovered transitions are the resets from every non-idle state back to idle. Covering them would require resetting the board in every possible state, which does not add functional insight.
- Overall file coverage: 94.44%

Achieving 100% statement and branch coverage with only six scenarios, while deliberately leaving reset transitions uncovered, demonstrates a targeted approach: scenarios are selected for their functional relevance rather than mechanical coverage maximisation.

2.4 Simulation Results

NOTE: In the following screenshots we cannot see the names of the signals, we didn't expand the window, please refer to the signal assignments in the corresponding test benches, located in the Appendix of this document.

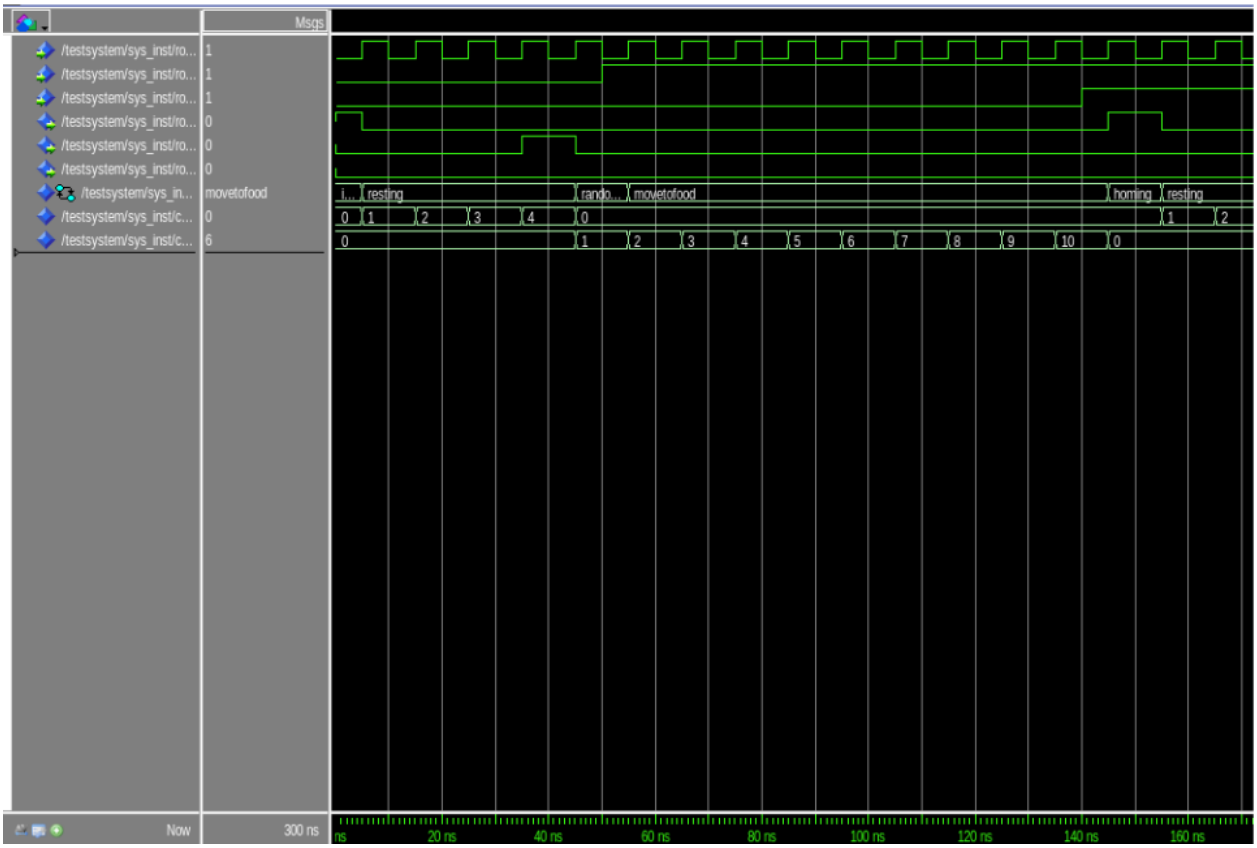
We will therefore show results of scenario 1 (because it is useful and needed to demonstrate results when PSL properties 1 and 2 are applied):



Simulation results of scenario 1

As seen in the chronogram, once state resting is reached, 4 clock cycles are waited in order to bypass the resting threshold and to reach state randomwalk, where the search threshold counter starts. After finding the food, between clock cycle 2 and 3 after the search threshold has started, we set close food to 1, therefore reaching state grabfood. From this point on, we do not rely on thresholds anymore. Subsequent signal assignments show state transitions from grabfood to resting, where we can see at the end of the screenshot that the resting threshold counter has restarted again from 1.

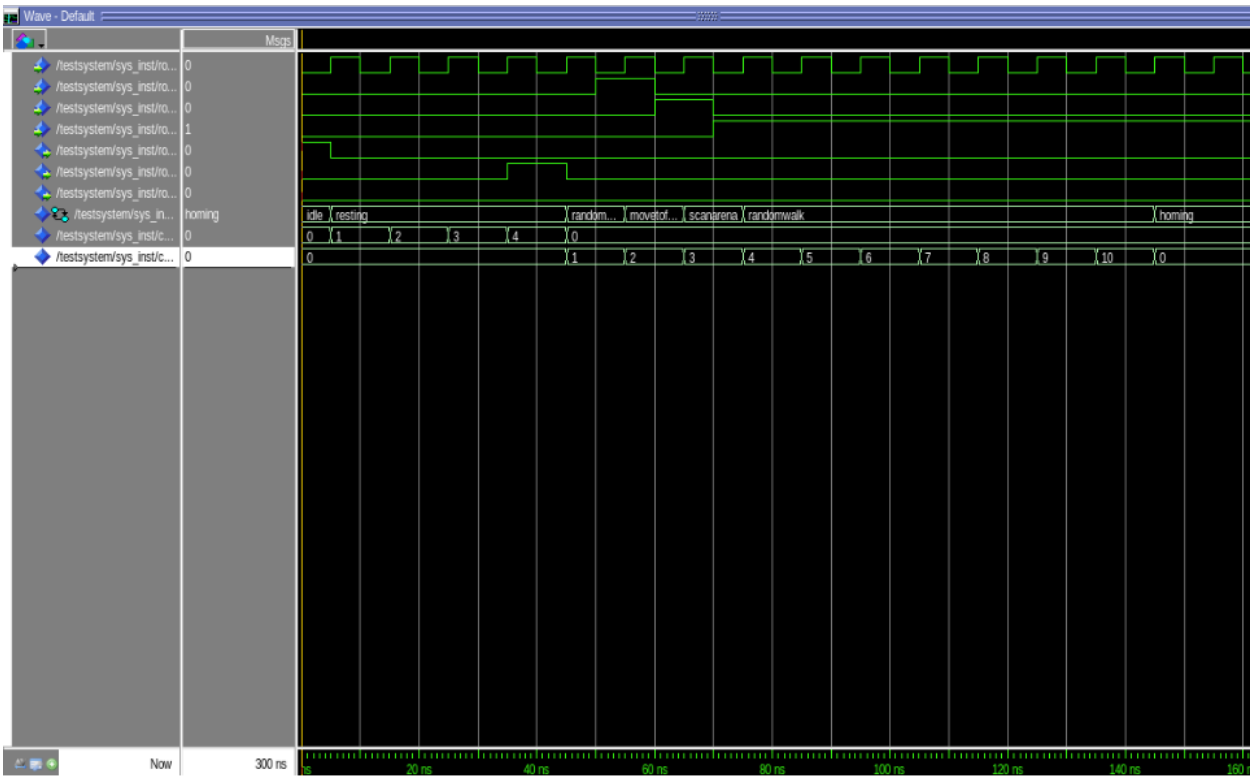
We now show simulation results of scenario 4:



Simulation results of scenario 4

As in the previous example above, when the rest threshold is reached, we transition to the randomwalk state and the counter of search starts since the output signal search is set to 1. At exactly 50ns, findfood is set to 1, and on the next rising edge, we move to the movetofood state. Then we wait for 10 clock cycles so that the search threshold exceeds. When state homing is reached, we set athome to 1 at 140ns, following the path towards state resting, where we restart again the rest threshold counter.

We show results of scenario 6 as well, because PSL property 3 is applied there:



Simulation results of scenario 6

The goal in this simulation is to demonstrate that state homing can be reached from randomwalk by setting scantimeup to 1 under correct conditions, meaning whilst in state scanarena and with the search counter not reaching 10. To reach this configuration, we need to access state movetofood, and this is done at 55ns, after setting findfood to 1 at exactly 50ns. Now what is left is to lose the food and set scantimeup to 1 between the interval [45ns, 145ns], which is achieved (scantimeup is set to 1 at 70ns and lostfood to 1 at 60ns, so that we reach state scanarena before reaching state randomwalk). The system behaves as expected!

3. ASIC Synthesis and Post-Synthesis Verification

3.1 Preparation of the Behavioural Description

Before submitting the VHDL to LeonardoSpectrum, the three classes of processes were reviewed against the requirements of synthesis:

- The next-state and output processes are purely combinational: every signal read appears in the sensitivity list, and every output is assigned in every branch. The synthesis tool maps them to combinational cells (AND, OR, NAND, NOR, MUX, AOI, OAI gates from the C35 CORELIB library).

- The state-register process is the only sequential element. Its structure (asynchronous reset followed by a synchronous clk'event assignment) maps unambiguously to D flip-flops with asynchronous reset (DFC1 cells in C35). No latches are inferred anywhere because all combinational processes have complete output assignments.

No modifications to the VHDL were required before synthesis.

3.2 Comparative Synthesis Results

The Robot controller was synthesised in three state-encoding configurations using LeonardoSpectrum with delay optimisation against the C35 0.35 µm CORELIB:

Encoding	Flip-flops	Area (µm²)	Max. Frequency (MHz)	Notes
Binary	4	3931	503.2	$\lceil \log_2(9) \rceil = 4$ bits; longer combinational paths
Gray	4	4313	478.5	Same bit count; slightly larger due to less optimised logic for non-sequential FSM
One-hot	9	4623	806.4	One FF per state; simpler decode logic, higher frequency

Binary encoding: Four flip-flops suffice to distinguish nine states ($2^4 = 16 > 9$). Each state transition requires the combinational logic to decode a 4-bit value and produce the next 4-bit code. The critical path traverses multiple levels of logic, limiting the frequency to 503.2 MHz.

Gray encoding: The tool assigns codes that differ by one bit between consecutive states. For a purely sequential counter this would minimise switching activity, but the Robot FSM has many non-sequential branches, so the benefit is limited. Area is slightly larger (4313 µm²) because the optimiser must handle more complex transition equations, and frequency is lower (478.5 MHz).

One-hot encoding: Nine flip-flops are used (one per state). Each state is identified by a single asserted bit, so the next-state and output logic becomes a simple OR of the relevant state bits, combined with the appropriate input conditions using AND. The number of logic levels is minimal, because of generated Boolean equations being much simpler, allowing a high frequency (806.4 MHz), due to the smaller critical path at the cost of a slightly larger area (4623 µm²).

These results are consistent with theoretical expectations. Binary minimises flip-flop count at the expense of combinational complexity. One-hot maximises speed at the cost of more registers. Gray offers little advantage for a non-sequential FSM. To sum up, one-hot encoding is the most suitable encoding for this FSM because it offers less combinational logic and higher performance, but at the cost of requiring more flip-flops.

One implementation detail surfaced during post-synthesis simulation: the synthesis tool connected the Q-bar output (inverted) of certain flip-flops. Specifically, in binary encoding, state_3 was inverted (represented by net nx536), and in one-hot encoding, state_6 and state_7 were inverted (nx773 and nx768). This had to be accounted for when adapting the PSL assertions and when reading simulation chronograms.

3.3 Post-Synthesis Simulation

Testbenches 1 and 6 were run against each of the three synthesised netlists. In all cases the primary output food and the state sequence matched the pre-synthesis simulation, confirming functional equivalence. The input assignments were adjusted to change at least one setup time before the rising clock edge, as required for post-synthesis timing, due to delta delay.

An important difference from pre-synthesis simulation is the state representation: instead of a symbolic state signal, the chronogram shows the individual flip-flop bits (state_0, state_1, …) or their complements (nx536, etc.). Reading the state requires knowing the precise encoding logged by LeonardoSpectrum. For example, in binary encoding state grabfood corresponds to state_3=0, state_2=1, state_1=1, state_0=0; since state_3 appears as nx536 (its complement), grabfood is visible as nx536=1, state_2=1, state_1=1, state_0=0. All three encodings were verified to match their respective encoding tables without any modification to the VHDL.

This approach of simulating without assertions, as easy as it can be, we cannot confirm yet whether the system works as expected. We need temporal PSL assertions to confirm the validity of the system, presented in the next section.

4. Validation with PSL Assertions

4.1 Role of Temporal Assertions in the Design Flow

Simulation with handwritten testbenches can only check the scenarios the designer thought to write. PSL assertions complement simulation by expressing properties that must hold for every sequence of inputs the testbench exercises. In our project, assertions served two distinct purposes: detecting a bug in the output process for rest (pre-synthesis), and verifying that post-synthesis netlists preserve the intended temporal relationships between inputs, states, and outputs.

4.2 The Three PSL Properties

All three properties use the PSL default clock, which defines the samples of the simulation with the rising edge of clk signal.

Property 1: Food cannot be grabbed before food is found.

always(search = '1' -> (state /= grabfood until (findfood = '1')))

We now explain the first assertion: When the robot starts its walk, i.e when the rest threshold exceeds, we know that search = 1. An implication is used because whenever search = 1, we are never in state grabfood, according to the FSM, which means that the right hand side can be evaluated immediately every time we start the search phase. Finally, we do not have a strong until, because findfood = 1 doesn’t need to occur in the execution trace in order to validate the assertion.

Property 2: Deposit always precedes the next rest signal.

always(state = grabfood -> (state = deposit before (rest = '1')))

For the same reasons here as well, we use an implication because we know that the rest signal could never be 1 before reaching state deposit IF we start from state grabfood, that is (because rest could be 1 before reaching state deposit if we start our path from state homing).

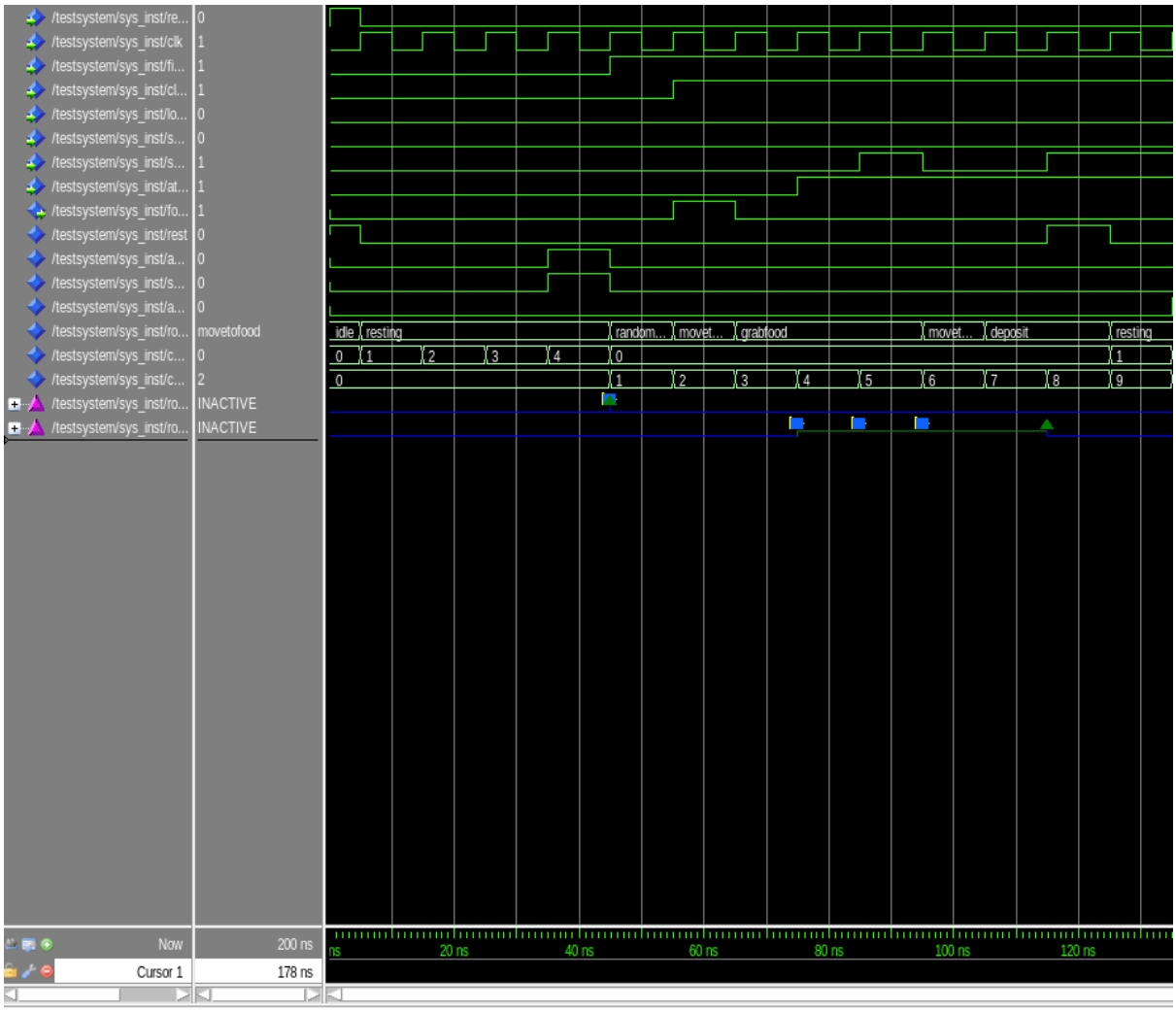
Property 3: After the full find-lose-scan sequence, the robot returns to randomwalk.

> ***always({state = randomwalk ; (abovesearchth = '0' and findfood = '0')[*]; (abovesearchth = '0' and findfood = '1'); (abovesearchth = '0' and lostfood = '0')[*]; (abovesearchth = '0' and lostfood = '1'); (abovesearchth = '0' and findfood = '0' and scantimeup = '0')[*]; (abovesearchth = '0' and findfood = '0' and scantimeup = '1')} |-> next!(state = randomwalk))***

What we have here is a typical SERE, because we are observing a sequence of events occurring. Starting from state randomwalk, one could reach state randomwalk again by doing a full loop by reaching movetofood (with findfood = 1), followed by a number of clock cycles where no event occurs (it may be 0 times), followed by reaching scanarena with lostfood = 1, with a possible number of clock cycles where we wait, until we set scantimeup = 1. Note that everything has to happen before the search threshold exceeds, otherwise this assertion would fail. We have an overlapping suffix implication because of SERE semantics, and because of that, we need to put a next! rule, because we know that the state would change at the NEXT clock cycle. And because we are sampling by clock rising edges, this works. The overlapping implication checks immediately the property on the right hand side, but in our case we need to wait until the state changes.

4.3 Simulation Results with PSL Assertions Pre-ASIC Synthesis

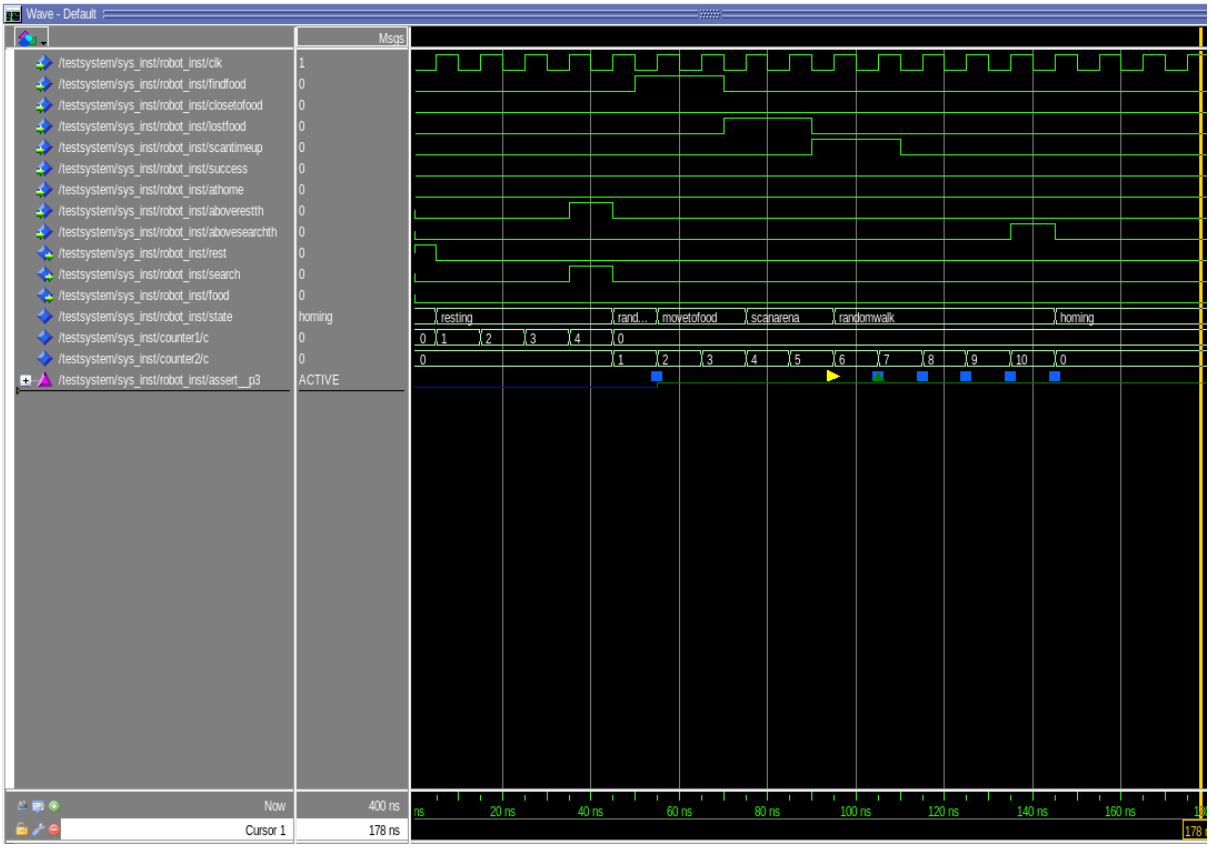
In this section, we display 2 chronograms with the results of the 3 PSL assertions introduced. We pick scenario 1 to display the validity of properties 1 and 2, and scenario 6 for property 3 verification.



Evaluating PSL properties 1 and 2 using scenario 1

The simulation results confirm the correctness of our system. Regarding the first property, because of the weak until rule we are using, the assertion is immediately evaluated and validated, because once the searching phase starts, we are not in state grabfood, but in randomwalk.

The second property was quite trickier for us to validate at first, not because the system was at fault, but because there was a timing issue in our test bench that led to having rest = 1 and state = deposit between 2 samples (or, in the same clock cycle). The solution was just to delay the assignment of success = 1, to have separate execution samples. In the chronogram above, it is obvious that rest = 1 after being in state deposit, because rest = 1 at 115ns, but we reach state deposit at 105ns.



Evaluating PSL property 3 using scenario 6

Regarding the evaluation of property 3, similarly, the third assertion was not passing and was constantly evaluating. This was fixed by adding more time between the transitions in the 6th testbench, increasing the time between events to 20ns instead of 10ns. The chronogram shows us the results: Between the interval of 10 clock cycles (where the search threshold counter is firing), we are passing through states movetofood and scanarena, an lastly, reaching state randomwalk with the search threshold not exceeded, findfood = 0 and scantimeup = 1.

4.4 Adaptation of PSL Assertions for Post-synthesis Netlists

After synthesis the symbolic state signal is replaced by individual bit signals. The PSL assertions must be rewritten to refer to these bits, including the inverted bits (nx536, nx773, nx768) where the tool used Q-bar outputs. The logical content of each property is identical. The only change is in the representation of the states, which are in bits now. For example, in binary encoding, state = randomwalk becomes nx536='1' and state_2='0' and state_1='1' and state_0='0'. All three properties passed on all three encodings after this mechanical adaptation, providing high confidence that the synthesis preserved the intended behaviour. Below, you can find the updated versions of the PSL assertions after ASIC synthesis.

Binary

```
  -- PSL property p1 is always(search = '1' -> ((nx536 /= '1' and state_2 /= '1' and
state_1 /= '1' and state_0 /= '0') until (findfood = '1')));

-- PSL property p2 is always((nx536 = '1' and state_2 = '1' and state_1 = '1' and state_0
= '0') -> ((nx536 = '0' and state_2 = '0' and state_1 = '0' and state_0 = '0') before
(rest = '1')));

-- PSL property p3 is always({(nx536 = '1' and state_2 = '0' and state_1 = '1' and
state_0 = '0') ; (abovesearchth = '0' and findfood = '0')[*]; (abovesearchth = '0' and
findfood = '1'); (abovesearchth = '0' and lostfood = '0')[*]; (abovesearchth = '0' and
lostfood = '1'); (abovesearchth = '0' and findfood = '0' and scantimeup = '0')[*];
(abovesearchth = '0' and findfood = '0' and scantimeup = '1')} |-> next!((nx536 = '1'
and state_2 = '0' and state_1 = '1' and state_0 = '0')));
```

Gray

```
  -- PSL property p1 is always(search = '1' -> ((state_3 /= '0' and state_2 /= '1' and
state_1 /= '0' and state_0 /= '1') until (findfood = '1')));

-- PSL property p2 is always((state_3 = '0' and state_2 = '1' and state_1 = '0' and
state_0 = '1') -> ((state_3 = '1' and state_2 = '1' and state_1 = '0' and state_0 = '0')
before (rest = '1')));

-- PSL property p3 is always({(state_3 = '0' and state_2 = '0' and state_1 = '1' and
state_0 = '1') ; (abovesearchth = '0' and findfood = '0')[*]; (abovesearchth = '0' and
findfood = '1'); (abovesearchth = '0' and lostfood = '0')[*]; (abovesearchth = '0' and
lostfood = '1'); (abovesearchth = '0' and findfood = '0' and scantimeup = '0')[*];
(abovesearchth = '0' and findfood = '0' and scantimeup = '1')} |-> next!((state_3 = '0'
and state_2 = '0' and state_1 = '1' and state_0 = '1')));
```

One-hot

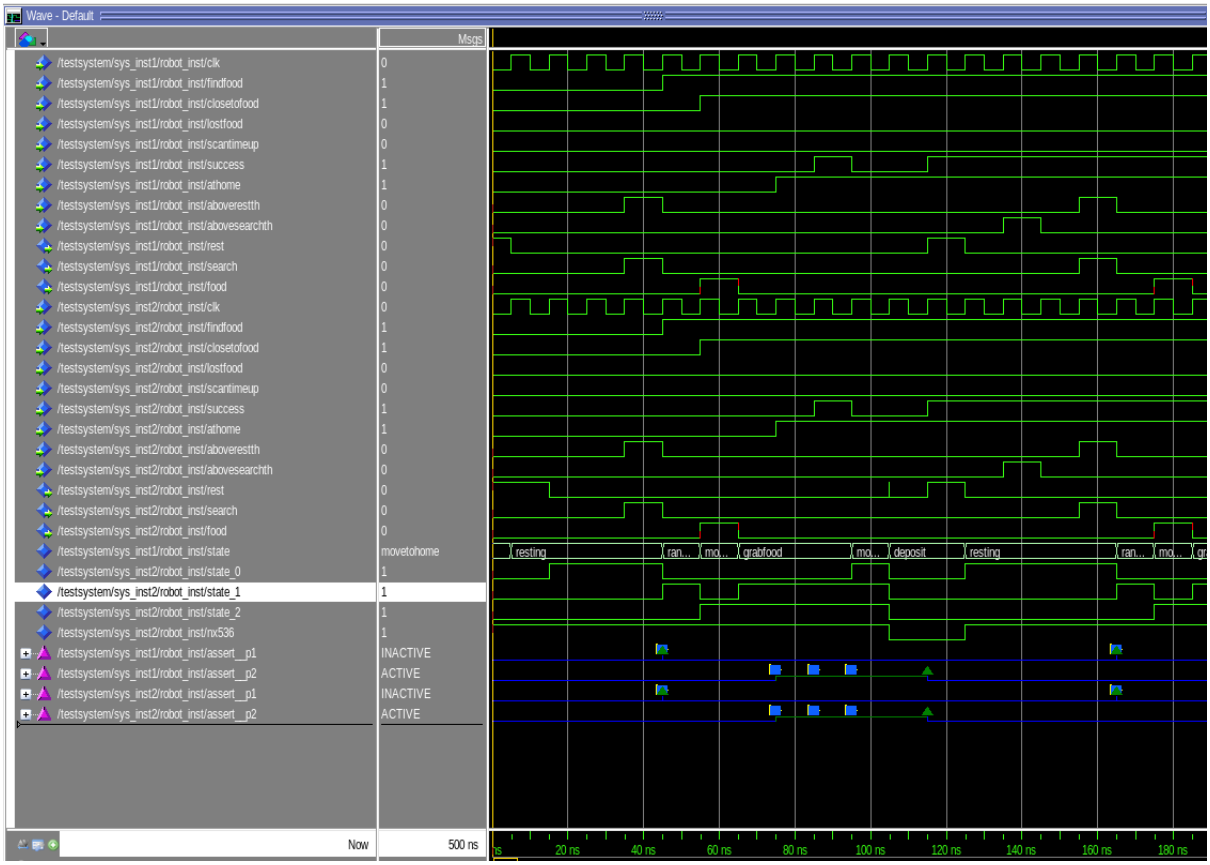
```
  -- PSL property p1 is always(search = '1' -> ((state_8 /= '0' and nx768 /= '1' and
nx773 /= '0' and state_5 /= '0' and state_4 /= '0' and state_3 /= '0' and state_2 /= '0'
and state_1 /= '0' and state_0 /= '0') until (findfood = '1')));

-- PSL property p2 is always((state_8 = '0' and nx768 = '1' and nx773 = '0' and state_5 =
'0' and state_4 = '0' and state_3 = '0' and state_2 = '0' and state_1 = '0' and state_0 =
'0') -> ((state_8 = '1' and nx768 = '1' and nx773 = '1' and state_5 = '0' and state_4 =
'0' and state_3 = '0' and state_2 = '0' and state_1 = '0' and state_0 = '0') before (rest
= '1')));

-- PSL property p3 is always({(state_8 = '0' and nx768 = '1' and nx773 = '1' and state_5
= '0' and state_4 = '0' and state_3 = '0' and state_2 = '1' and state_1 = '0' and state_0
= '0') ; (abovesearchth = '0' and findfood = '0')[*]; (abovesearchth = '0' and findfood =
'1'); (abovesearchth = '0' and lostfood = '0')[*]; (abovesearchth = '0' and lostfood =
'1'); (abovesearchth = '0' and findfood = '0' and scantimeup = '0')[*]; (abovesearchth
= '0' and findfood = '0' and scantimeup = '1')} |-> next!((state_8 = '0' and nx768 = '1'
and nx773 = '1' and state_5 = '0' and state_4 = '0' and state_3 = '0' and state_2 = '1'
and state_1 = '0' and state_0 = '0')));
```

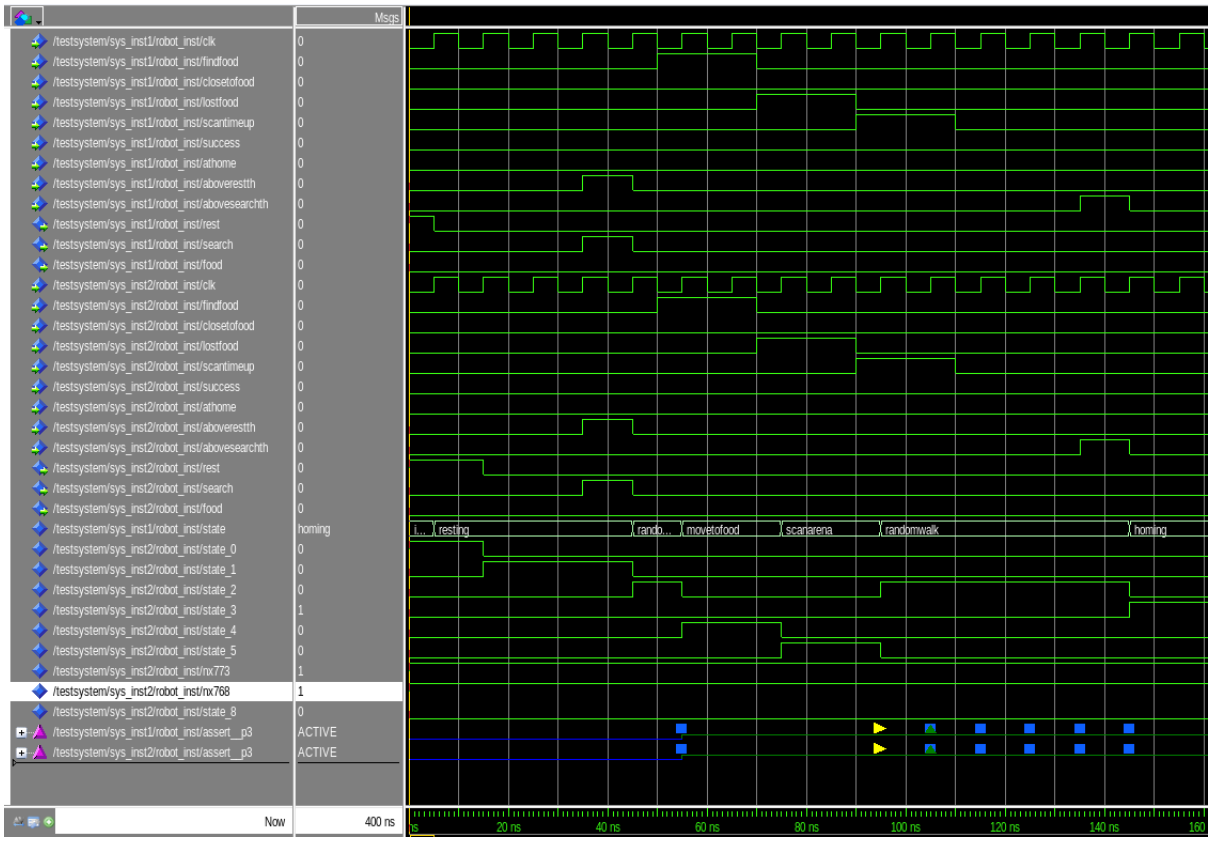
4.5 Simulation Results after adapting the PSL Assertions for Post-synthesis Netlists

Because we are in a post-ASIC synthesis phase, the chronograms will be much different compared to the usual ones seen previously. Depending on which type of encoding is chosen, with the fact that we encode the states and the primary outputs, a lot more signals will be present in the chronograms. Together, with the PSL assertions mentioned and adapted, we are presenting 2 chronograms (the rest are in the appendix) for demonstrational purposes, where we test the validity of the adapted post-ASIC synthesis PSL assertions, while using the previous, unencoded pre-synthesis results as a reference. Because we use configurations to facilitate the distinguishability between which robot instance is used, setting up these tests was easy.



Simulation of testbench 1, using binary encoding in the controller post synthesis

As an example, we choose to simulate testbench 1 using binary encoding. Instance 1 of the robot represents the robot pre-synthesis, while instance 2 represents the robot post-synthesis. Note the state encodings at the bottom of the screenshot: Because the synthesis tool used Q-bar to represent state_3 signal, we have a new signal that represents the opposite of what it should be. Thankfully, with our carefully adapted PSL assertions (considering the Q-bar signal(s)), the PSL assertions for property 1 and 2 are passing, with the execution traces being the same between instance 1 and 2, proving that the robot behaves as expected post-ASIC synthesis.



Simulation of testbench 6, using one-hot encoding in the controller post synthesis

In this chronogram, we present the execution of test bench 6, using one-hot encoding. Like previously, instance 1 represents the robot pre-synthesis, and instance 2 in post-synthesis. In this case, the tool presents signals state_6 and state_7 with Q-bar. We effectively have 9 bits to represent a state, knowing we have 9 states, so in that regard the encoding is correct. Together, with our adapted PSL assertions in regard with the specific state encodings, we can see that the execution traces are equal for instance 1 and 2, validating property 3 in the post-synthesis phase. The robot works as expected at the netlist level!

5. FPGA Synthesis

5.1 Adaptations to the VHDL Description

Moving from ASIC to FPGA required several concrete changes to the VHDL:

Active-low reset: The DE1 board's push-button is active-low. All processes were modified to check reset = '0' instead of reset = '1'. Since the simulation testbench uses active-high reset, the polarity inversion must be handled carefully in the FPGA-specific version.

Clock division: The board's 50 MHz quartz is too fast for human observation of counter values. A clock-divider process using a local variable cnt (integer range 0 to 67108863) counts 50,000,000 cycles and generates a 1 Hz pulse ck_1Hz. This signal drives Robot and both counters.

I/O port mapping: The System entity was extended with LEDR(5 downto 0), LEDG(2 downto 0), seg_val_hex0/1/2 (INTEGER range 0 to 127). Chip-pin attributes bind each signal to the

physical DE1 pins (R22 for reset, L1 for the 50 MHz clock, U12–L22 for switches, V19–R20 for red LEDs, etc.).

Seven-segment display encoding: Each FSM state is assigned a digit 0–8 displayed on HEX0. The binary patterns (e.g., 2#0000001# for '0', 2#1001111# for '1') drive the seven active-high segments of the display. Counter values use the same encoding on HEX1 and HEX2.

Counter type restriction: The counter's internal variable c was changed from natural (unbounded) to natural range 0 to 15, allowing Quartus to infer exactly 4 flip-flops for the counter register.

5.2 Synthesis Results

Quartus II compiled the System entity successfully for the EP2C20F484C7 (Cyclone II) device:

Resource	Available	Used	Utilisation
Logic Elements (CLBs)	18,752	92	< 1%
LUTs	18,752	89	< 1%
Dedicated registers (flip-flops)	18,752	44	< 1%

On the Cyclone II, each logic element contains a LUT and a flip-flop, so logic element count equals CLB count. The 44 flip-flops are accounted for as follows:

- Clock divider: The variable cnt: integer range 0 to 67108863 requires $\lceil \log_2(67{,}108{,}864) \rceil$ = 26 flip-flops.
- Counter 1 (rest): 4 flip-flops for c (natural range 0 to 15) + 1 for the counting/idle state bit = 5 flip-flops.
- Counter 2 (search): same, 5 flip-flops.
- Robot FSM: Quartus uses one-hot encoding by default for FPGAs (as confirmed in the RTL Viewer). Nine states should give 9 flip-flops, but Quartus optimises away the idle state register (since idle = all zeros in one-hot), yielding 8 flip-flops.
- Total: 26 + 5 + 5 + 8 = 44.

Comparing with the ASIC one-hot synthesis: the ASIC used 9 flip-flops for the Robot alone (no clock divider, unbounded counter). Removing the clock divider from the FPGA count gives 18 flip-flops (5 + 5 + 8), versus the ASIC's 9 + 5 + 5 = 19 (one-hot + two counters). The difference of one flip-flop is precisely the Quartus optimisation of the idle state.

On-board testing confirmed correct behaviour: switches toggled the corresponding red LEDs, state transitions were visible on HEX0, and counter values advanced on HEX1 and HEX2 at a 1 Hz rate. The active-low reset correctly reinitialised the FSM to idle (displaying '0' on HEX0) at any point.

5.3 FPGA simulations

Several executions were run in the FPGA board, in order to check its correctness. The FPGA was configured as such, with the hope that it matches the formal specification at the beginning stage. Because a limited number of scenarios are possible in a small FSM like ours, we followed the same

philosophy of testing as in the test benches described earlier. Of course, the test was done by the client, which in this case was our professor.

Test	Scenario	Comment/Conclusion on FPGA execution
1	idle → resting → randomwalk → movetofood → grabfood → movetohome → deposit → resting	On this particular test, we had to wait physically 4 seconds so that we could achieve the state randomwalk. Then, because of the searching threshold of 10 seconds, during that interval we had to find the food and then grab it, to achieve state grabfood, where no thresholds are used from that point on.
2	resting → randomwalk → homing → resting	Here, we physically wait 14 seconds (resting + searching thresholds bypassed) to reach state homing, in which case we pressed the button corresponding to athome = 1
3	resting → randomwalk → movetofood → scanarena → homing	Similarly, we waited for 4 seconds due to the resting threshold, found the food, but was unable to grab it in time for 10 additional seconds, which bypassed the searching threshold and moving us to state homing
4	resting → randomwalk → movetofood → homing → resting	Similar to test 2
5	resting → randomwalk → movetofood → scanarena → movetofood → homing	Also similar to test 2, here we tried alternating between movetofood and scanarena fast in a period of 10 seconds, before going to homing
6	resting → randomwalk → movetofood → scanarena → randomwalk → homing	Here, we tested the input scantimeup during the 10 second interval to check if we would go to state randomwalk, which we did succeed.

One thing to note here is that simulating using the FPGA board, compared to post ASIC synthesis, was a lot easier to interpret, due to the fact that we had the physical board beside us. However, this does not imply that precise simulations could have been made, because these types of simulations rely on humans, and humans are not prone to errors (for instance forgetting to press a button on time, etc). Post ASIC synthesis simulations were harder to read, firstly because chronogrammes in general are not easy to interpret, and second because a lot more signals were present this time due to the state and primary outputs encoding! While these simulations provide precision and insight, understanding them carefully may be more challenging. No bugs were detected during FPGA simulations.

6. Conclusions

This project has been a complete end-to-end illustration of the RT-level design flow. Several lessons emerge from the experience:

Verification drives design quality. Two concrete bugs were found and fixed thanks to verification artefacts: the PSL property 2 (rest before deposit) revealed a timing assumption error in the testbench that would have been invisible without the formal assertion; property 3 required adjusting event timing to align with the clock sampling discipline. Neither error would have been caught by a purely visual inspection of chronograms.

Coverage is a tool, not a goal. Reaching 100% statement and branch coverage with targeted scenarios is more informative than reaching 66% transition coverage by adding reset-from-every-state tests. We must understand which uncovered items are acceptable and which indicate gaps in the test suite.

Synthesis is generally faithful, but details matter. Post-synthesis simulation matched pre-synthesis simulation in all six scenarios for all three encodings without a single functional discrepancy. However, the Q-bar optimization for certain flip-flops required careful adaptation of PSL assertions. This is a subtle but important issue in real industrial flows.

State encoding is a genuine engineering trade-off. The experimental data confirms theory: binary minimises area and flip-flop count; one-hot nearly doubles the frequency (806 MHz vs 503 MHz) at the cost of more registers and slightly larger total area. For a performance-critical controller in a modern ASIC process where register area is cheap, one-hot is the natural choice.

FPGA and ASIC flows share a common root but diverge in practice. The same VHDL source (with minor adaptations) produced correct results on both targets. The main differences are the clock-division requirement for FPGA, the active-low reset convention, and the physical pin-mapping mechanism. Quartus and LeonardoSpectrum independently applied one-hot encoding, arriving at compatible flip-flop counts.

More broadly, the project shows that the RT-level design flow gives a clear and structured way to go from a specification to working hardware. Each step produces results that help check and improve the next one. For example, testbenches help find functional bugs, coverage reports show what parts of the design have not been tested properly, assertions reveal timing assumptions that are not visible in chronograms, and synthesis logs show implementation details that are hidden in the RTL model. All of this helps reduce the risk of unexpected problems later in the design process. At the same time, the flow also has some limitations. PSL assertions checked during simulation can only verify behavior for the testbench scenarios that are actually run. If a property is never triggered, it does not really prove anything about the design.

Overall, we started really slowly with this project, but we think that the progress we made incrementally with each lab was significant. It shows that with constant work and logical reasoning, everything becomes easier.

Appendix

The following excerpts are provided for reference. Full source files were submitted with each lab session.

A.1 Robot Entity and Architecture (pre-synthesis)

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity Robot is
       port(reset, clk,
       findfood, -- location of the food has been identified
       closetofood, -- moving towards the food
       lostfood, -- location of the food has been lost
       scantimeup, -- scan time has expired
       success, -- the food has been grabbed, or deposited
       athome, -- control is given to exit the arena
       aboverestth, abovesearchth : in std_logic;
       rest, search, food : out std_logic);
end Robot;

architecture intf of Robot is
type States is (idle, resting, randomwalk, homing, movetofood, scanarena, grabfood,
movetohome, deposit);
signal state, nextstate: States;

    -- PSL default clock is (clk'event and clk = '1');

    -- PSL property p1 is always(search = '1' -> (state /= grabfood until! (findfood =
'1')));

    -- PSL property p2 is always(state = grabfood -> (state = deposit before! (rest =
'1')));

    -- PSL property p3 is always({ state=randomwalk; {abovesearchth='0' and
findfood='0'}[*]; {abovesearchth='0' and findfood='1'}; {abovesearchth='0' and
lostfood='0'}[*]; {abovesearchth='0' and lostfood='1'}; {abovesearchth='0' and
findfood='0' and scantimeup='0'}[*]; {abovesearchth='0' and findfood='0' and
scantimeup='1'}}|-> next! state=randomwalk);

    -- PSL assert p1;
    -- PSL assert p2;
```

```
    -- PSL assert p3;

begin
    -- next state
    process(state, findfood, closetofood, lostfood, scantimeup, success, athome,
aboverestth, abovesearchth)
    begin
       case state is
              when idle => nextstate <= resting;
              when resting =>
                     if aboverestth = '1' then nextstate <= randomwalk;
                     else nextstate <= resting;
                     end if;
              when randomwalk =>
                     if (abovesearchth = '1') then nextstate <= homing;
                     elsif (findfood = '1') then nextstate <= movetofood;
                     else nextstate <= randomwalk;
                     end if;
              when movetofood =>
                     if (abovesearchth = '1') then nextstate <= homing;
                     elsif (closetofood = '1')  then nextstate <= grabfood;
                     elsif (lostfood = '1')  then nextstate <= scanarena;
                     else nextstate <= movetofood;
                     end if;
              when scanarena =>
                     if (abovesearchth = '1') then nextstate <= homing;
                     elsif (findfood = '1') then nextstate <= movetofood;
                     elsif (scantimeup = '1') then nextstate <= randomwalk;
                     else nextstate <= scanarena;
                     end if;
              when homing =>
                     if athome = '1' then nextstate <= resting;
                     else nextstate <= homing;
                     end if;
              when grabfood =>
                     if success = '1' then nextstate <= movetohome;
                     else nextstate <= grabfood;
                     end if;
              when movetohome =>
                     if athome = '1' then nextstate <= deposit;
                     else nextstate <= movetohome;
                     end if;
              when deposit =>
                     if success = '1' then nextstate <= resting;
                     else nextstate <= deposit;
                     end if;
       end case;
    end process;

    -- updating of the state register:
    process(reset, clk)
```

```
    begin
       -- asynchronous reset (active high):
       if (reset = '1') then state <= idle;
       -- otherwise, on clock rising edges:
       elsif (clk'event and clk='1') then
              state <= nextstate;
       end if;
    end process;

    -- output function for rest
    process(state, success, athome)
    begin
       if (state=idle) then rest <= '1';
       elsif (state=homing) and (athome='1') then rest <= '1';
       elsif (state=deposit) and (success='1') then rest <= '1';
       else rest <= '0';
       end if;
    end process;

    -- output function for food
    process(state, closetofood, abovesearchth, lostfood)
    begin
       if (state=movetofood) and (abovesearchth='0') and (lostfood = '0') and
(closetofood='1') then food <= '1';
       else food <= '0';
       end if;
    end process;

    -- output function for search
    process(state, aboverestth)
    begin
       if (state=resting) and (aboverestth='1') then search <= '1';
       else search <= '0';
       end if;
    end process;
end intf;
```

A.2 LeonardoSpectrum Synthesis Summary

Metric	Binary	Gray	One-hot
Flip-flops (DFC1/DFP1)	4	4	9
Total area (µm²)	3931	4313	4623
Max. frequency (MHz)	503.2	478.5	806.4
State bits	4	4	9

A.3 FPGA Compilation Summary (Quartus II, EP2C20F484C7)

Resource	Used / Available
Logic Elements	92 / 18,752 (< 1%)
LUTs	89 / 18,752 (< 1%)
Registers	44 / 18,752 (< 1%)

A.4 Testbench code from lab 3

TEST 1

```
begin
-- system instqntiation
sys_inst: System port map(reset, clk, findfood, closetofood,
lostfood, scantimeup, success, athome, food);
reset <= '1', '0' after 5 ns;
clk <= not clk after 5 ns;
-- this is a simulation example where we want to check the appearance
of 1 from food. then we go back to the resting point.
findfood <= '1' after 50 ns;
closetofood <= '1' after 60 ns;
--lostfood <=
--scantimeup <=
success <= '1' after 70 ns, '0' after 80 ns, '1' after 90 ns;
athome <= '1' after 80 ns;
end test1;

```

TEST 4

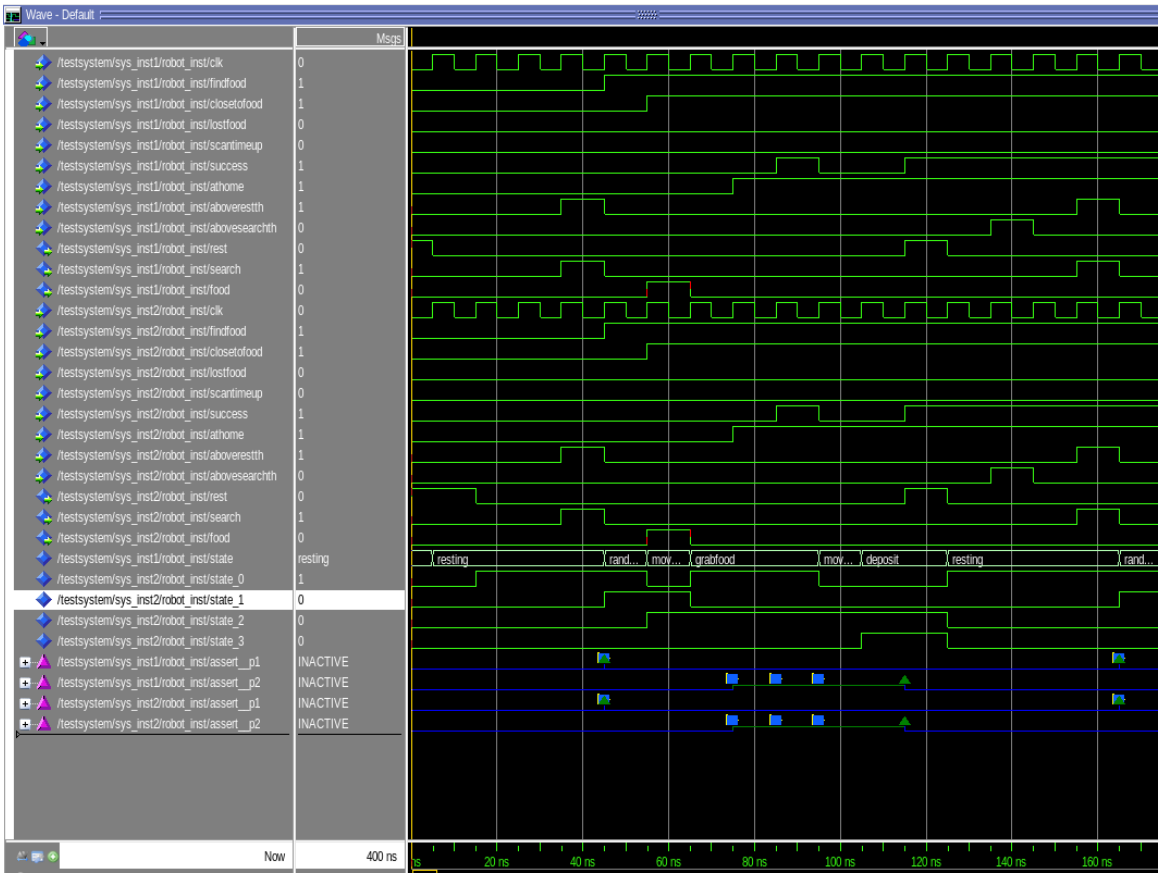
```
begin
-- system instqntiation
sys_inst: System port map(reset, clk, findfood, closetofood,
lostfood, scantimeup, success, athome, food);
6
reset <= '1', '0' after 5 ns;
clk <= not clk after 5 ns;
-- this is a simulation example where we want to check the transition
between movetofood and homing due to the activation of the
abovesearchth threshold and returning to resting when athome = 1.
findfood <= '1' after 50 ns;
athome <= '1' after 140 ns;
end test4;

```

TEST 6

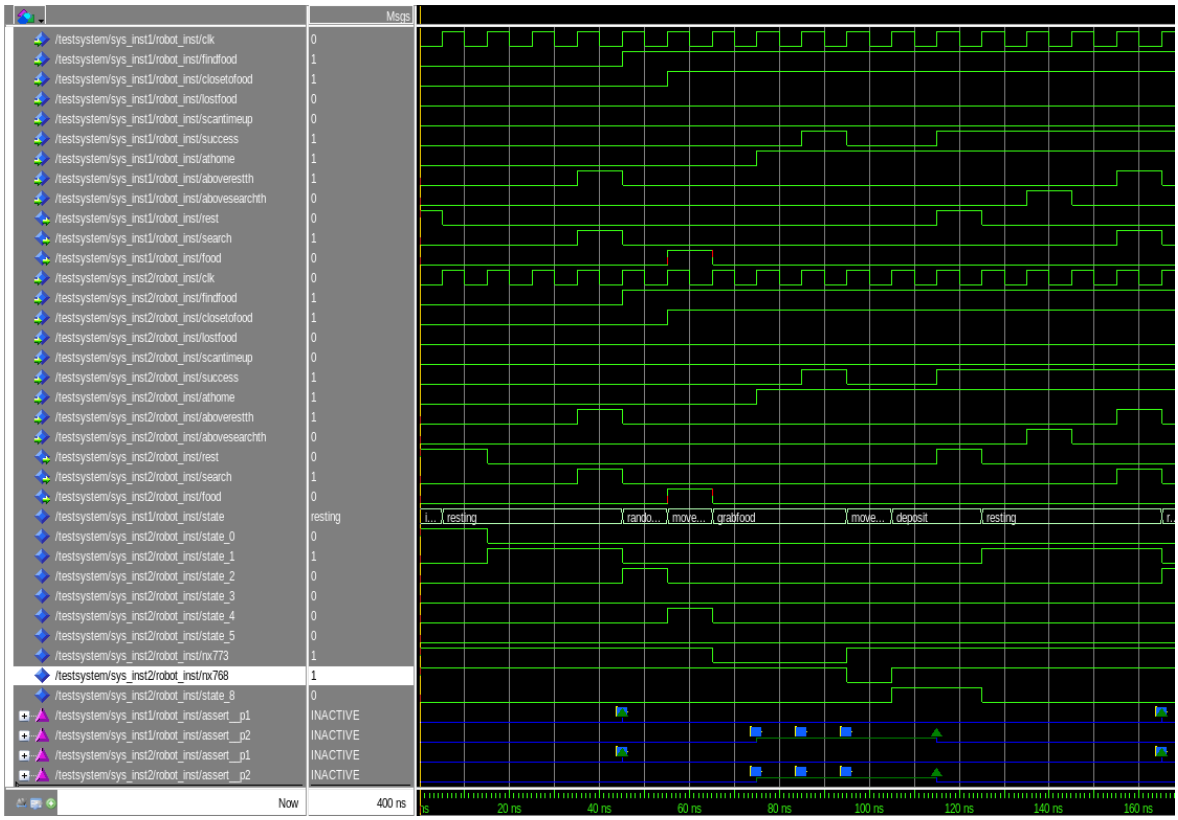
```
  begin
-- system instqntiation
sys_inst: System port map(reset, clk, findfood, closetofood,
lostfood, scantimeup, success, athome, food);
reset <= '1', '0' after 5 ns;
clk <= not clk after 5 ns;
-- this is a simulation example where we want to check the transition
scanarena to randomwalk state when the signal scantimeup is set to 1.
```

```
findfood <= '1' after 50 ns, '0' after 60 ns;
lostfood <= '1' after 60 ns, '0' after 70 ns;
scantimeup <= '1' after 70 ns;
end test6;
```

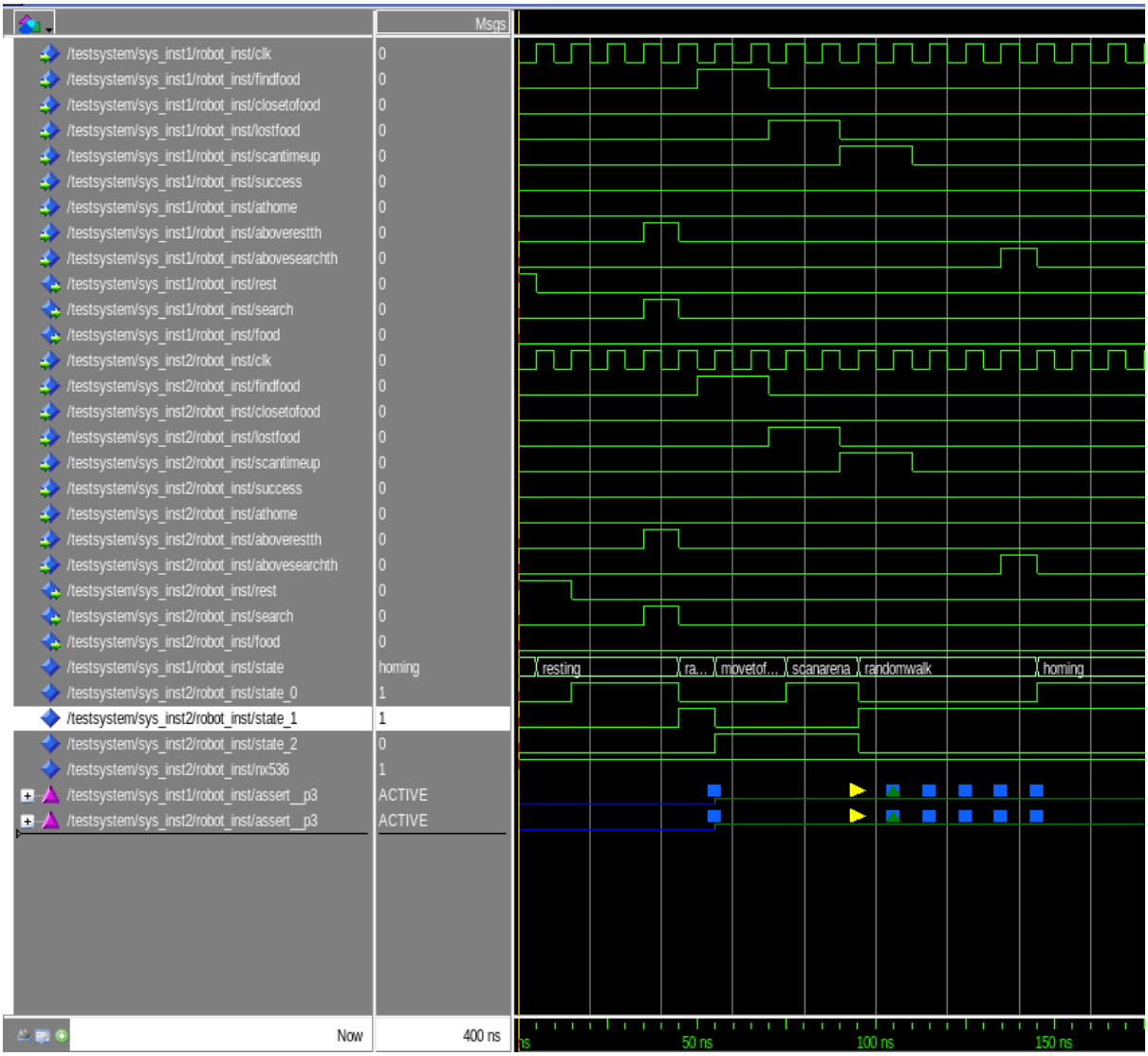


A.5 Post-ASIC synthesis chronograms, using different types of state encodings

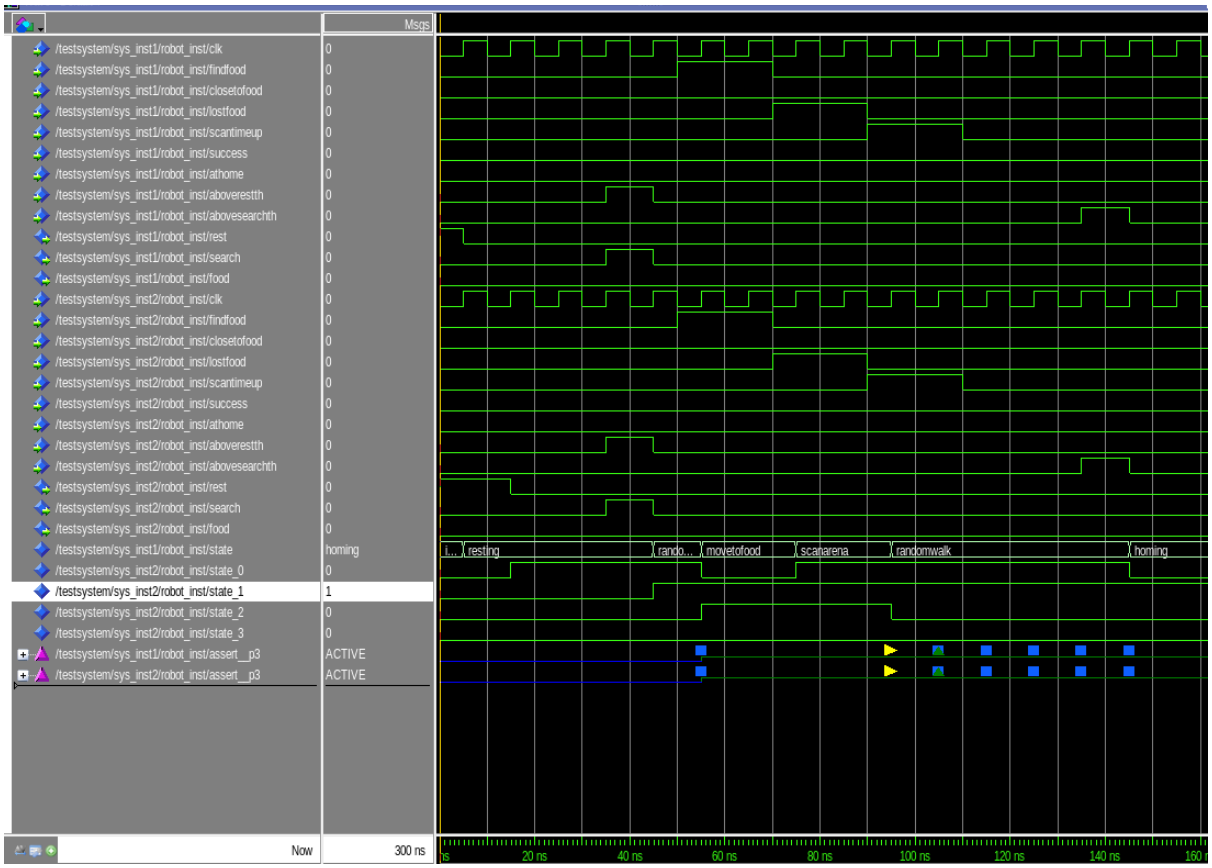
Simulation of testbench 1, using gray encoding in the controller post synthesis



Simulation of testbench 1, using one-hot encoding in the controller post synthesis



Simulation of testbench 6, using binary encoding in the controller post synthesis



Simulation of testbench 6, using gray encoding in the controller post synthesis